

M7. pandas 시계열 도구

시트로는 어려운 일들, 오늘은 이걸 해본다

M6 연결 — 시트 흉내는 끝났다

- M6에서 pandas로 CSV를 불러오고, M1-M3(그래프·평균·이동평균)을 코드로 재현했다.
- 사실 그건 시트가 하던 일을 코드로 옮긴 것뿐이었다.
- 오늘은 시트로써 어렵거나 아주 번거로운 일들이다. 열쇠는 하나 — 날짜를 행의 이름표로 만드는 것.

오늘의 목표

- 날짜를 인덱스로 만들고, `loc` 으로 날짜로 행을 찾을 수 있다.
- `resample` 로 시간 단위를 바꿀 수 있다(일별 → 월별·연별).
- `rolling` 의 `center=True` 옵션과 결측치(NaN) 다루는 법을 안다.

날짜 인덱스 — 열쇠 하나

- 지금까지 `df` 의 행은 0, 1, 2... 번호로 불렸다.
- `index_col="date"` 를 추가하면 date 열이 각 행의 이름표(인덱스)가 된다.
- 이제 행을 번호가 아니라 날짜로 부를 수 있다.

```
import pandas as pd

url = "https://raw.githubusercontent.com/jaehoonkim/timeseries-academy/main/data/seoul-temp-daily.csv"
df = pd.read_csv(url, parse_dates=["date"], index_col="date")
df.head()
```

M6의 불러오기 코드에 `index_col` 하나만 더한 것이다.

날짜로 부르기 — loc 읽는 법

```
df.loc["2023-08-10"]
```

= 이름표(인덱스)가 "2023-08-10"인 행을 찾아 달라

(loc = location)

코드	고르는 것
<code>df["temp_avg"]</code>	열(세로)
<code>df.loc["날짜"]</code>	행(가로), 이름표로 찾는다

"2026-01"처럼 달까지만 쓰면 그 달의 모든 행이 나온다.

loc으로 조회하기

하루를 콕 짚거나, 달을 통째로 부를 수 있다.

```
df.loc["2023-08-10"]
```

```
df.loc["2026-01"]
```

결과는 직접 실행해서 확인해 보자.

구간과 계산 — loc에 슬라이싱까지

고른 결과에 함수를 이어 붙일 수도 있고, 콜론(:)으로 날짜 범위를 자를 수도 있다.

```
df.loc["2026-01"]["temp_avg"].mean()
```

```
df.loc["2026-01-20":"2026-01-22"]
```

M5에서 "몇 번째 행인지" 세던 일이, 이제는 날짜를 그대로 쓰는 일이 됐다.

resample — 시간 단위를 바꾸는 도구

M4에서 월별 평균을 내려면 세 단계였다: `=MONTH()` 로 월 열 만들기 → 피벗에서 행=월로 묶기 → 값을 AVERAGE로 바꾸기.

```
df["temp_avg"].resample("ME").mean()
```

= temp_avg를 / 월 단위로 다시 묶어서 / 묶음마다 평균

- `"ME"` 는 Month End의 약자 — 월 단위로 묶고 월말 날짜로 라벨을 붙인다.
- `.max()` 로 바꾸면 월별 최고값이 된다.

resample 실행해보기

```
monthly = df["temp_avg"].resample("ME").mean()  
monthly
```

```
monthly.plot()
```

세 단계였던 일이 한 줄이 됐다. 값이 몇 개 나오는지 활동지에서 직접 세어 보자.

뭉치기 전엔 count부터

resample은 강력하지만, 몇 개를 뭉쳤는지는 말해 주지 않는다. 연단위로도 뭉쳐 보자.

```
df["temp_avg"].resample("YE").mean()
```

```
df["temp_avg"].resample("YE").count()
```

숫자가 이상해 보이면, 뭉친 값을 믿기 전에 먼저 몇 개를 뭉쳤는지 확인하는 습관을 들이자.

rolling의 숨은 옵션 — center=True

- `rolling(window=7)` 은 항상 그날까지의 7일(뒤로 난 창)을 본다 — 그래서 M3에서 이동평균이 급한 변화를 며칠 늦게 따라갔다.
- `center=True` 는 창의 가운데에 그날을 놓는다(앞 3일 + 그날 + 뒤 3일).
- 그 대가로, 창의 뒤쪽 절반은 아직 오지 않은 데이터가 있어야 계산된다.

center=True 실행해보기

trailing과 centered를 나란히 놓고 비교해 보자.

```
avg = df["temp_avg"]
compare = pd.DataFrame({
    "actual": avg,
    "trailing": avg.rolling(window=7).mean(),
    "centered": avg.rolling(window=7, center=True).mean(),
})
compare.loc["2023-08"].plot()
```

그래프에서 어느 쪽이 실제(actual)와 더 가까운지, 그리고 이 창을 예측에 써도 될지는 활동지에서 생각해 보자.

NaN — 값 없음은 에러가 아니다

- 표에 값이 없는 자리는 **NaN**(Not a Number)으로 표시된다. 코드에서 `None` 을 넣으면 pandas가 NaN으로 저장한다.
- NaN은 에러가 아니라 "여기는 값이 없다"는 표시다.
- 실제 데이터에는 이런 구멍(결측)이 흔하다. 시나리오: 태풍에 관측 장비가 사흘 고장 났다면?

```
broken = df["temp_avg"].copy()  
broken.loc["2023-08-10":"2023-08-12"] = None  
broken.isna().sum()
```

`isna().sum()` 은 구멍의 개수를 센다.

결측치 채우기 — interpolate

```
broken.interpolate()
```

= broken의 구멍마다 / 양옆에 남은 값을 직선으로 이어 /
구멍 자리의 값을 그 선 위에서 읽어 채우기

```
fixed = broken.interpolate()  
fixed.loc["2023-08-09":"2023-08-13"]
```

채운 값이 실제와 얼마나 비슷한지는 활동지에서 직접 비교해 보자.

결측치 다루는 선택지

결측을 만나면 선택지가 여럿이다.

- 값을 채운다 — `interpolate` (양옆을 직선으로 이어 추측)
- 그 행을 지운다 — 대신 시간 간격이 어떻게 되는지 생각해 봐야 한다
- 채우지 않고 NaN인 채로 두되, 구멍이라는 표시는 남긴다

어떤 선택이 맞는지는 상황에 따라 다르다 — 활동지에서 직접 따져 보자.

오늘 배운 세 문장

1. 날짜가 인덱스가 되면 pandas는 시계열 전용 도구가 된다.
2. 뭉치기 전에 count부터.
3. 채우기 전에 왜 비었는지부터.

이제 활동지로

1. 날짜 인덱스를 만들고 `loc` 으로 날짜를 불러본다.
2. `resample` 로 월별.연별로 뭉쳐 보고, 합정을 확인한다.
3. `center=True` 로 `trailing`과 `centered`를 비교한다.
4. 결측치를 만들고, 채우고, 무엇을 조심해야 할지 생각해 본다.